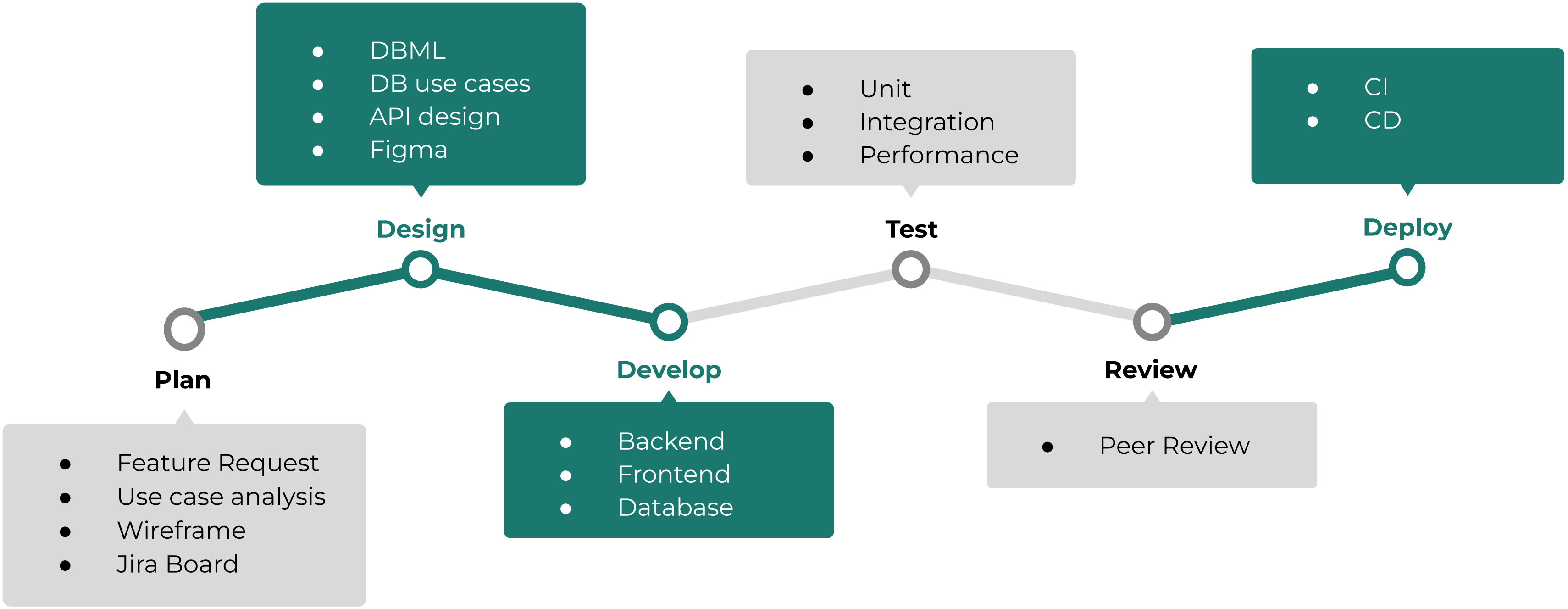
Comprehensive Overview of ERP Cloud Software Development

A Collaborative Session

Project Management

Project Management



Plan

- Group Discussion
- Wireframing

Use case analysis

Feature Request

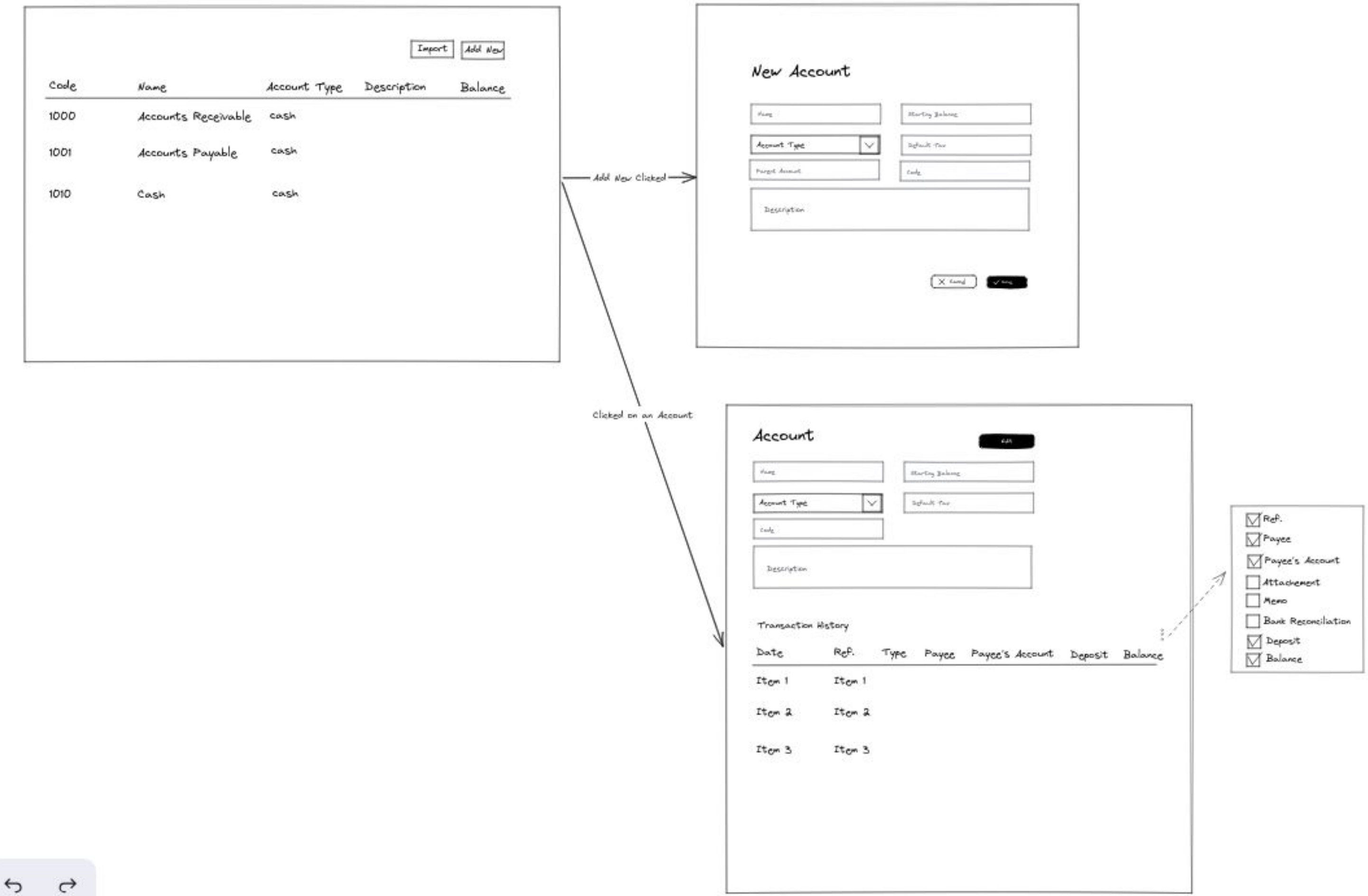
- Product Team
- Business Team

Chart of Accounts

Chart of Accounts*

Account:

- Code
- Account Type
- Name
- Description
- Balance



30%

Plan

- Group Discussion
- Wireframing

Use case analysis

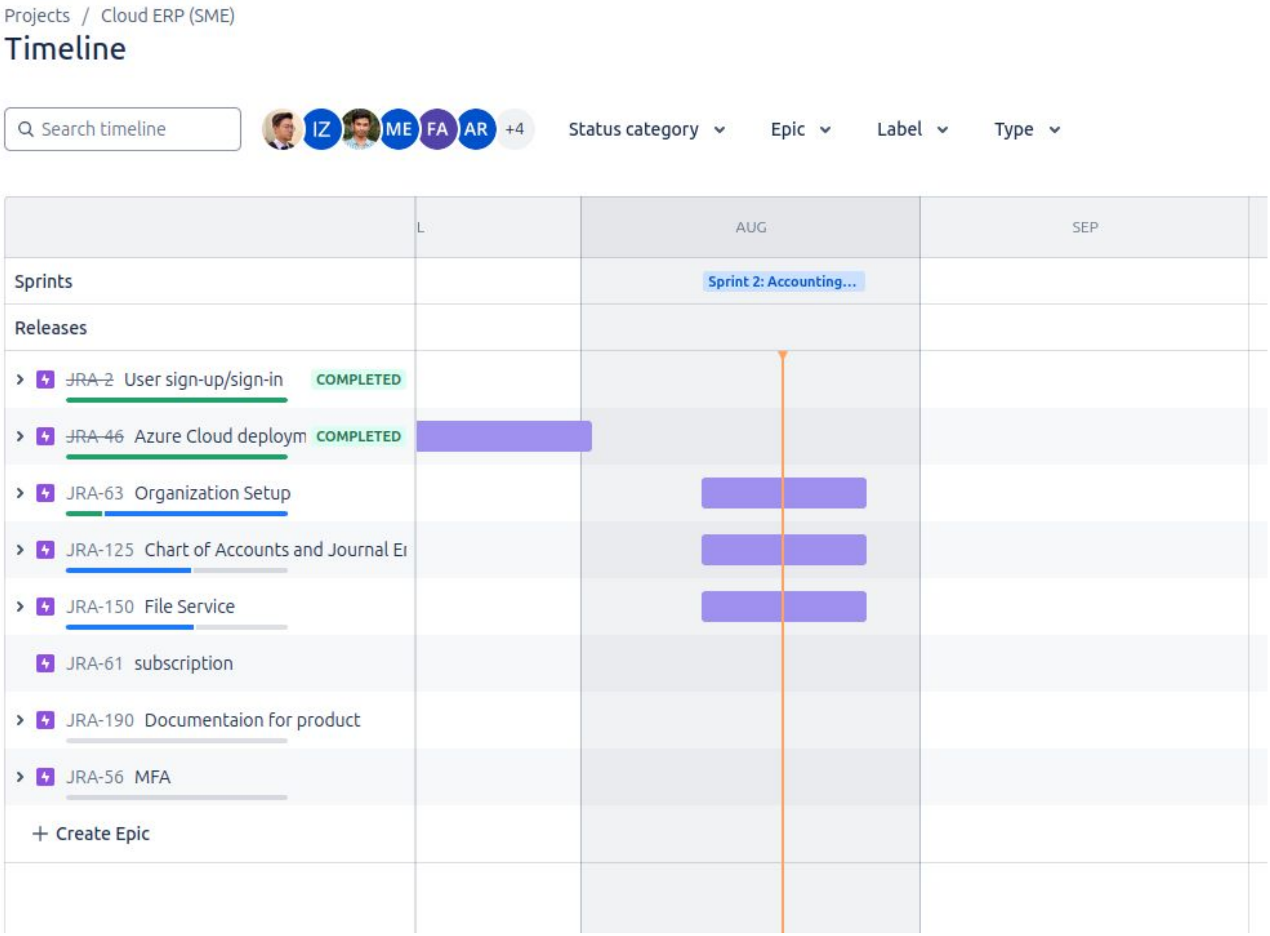
Feature Request

- Product Team
- Business Team

Jira Board

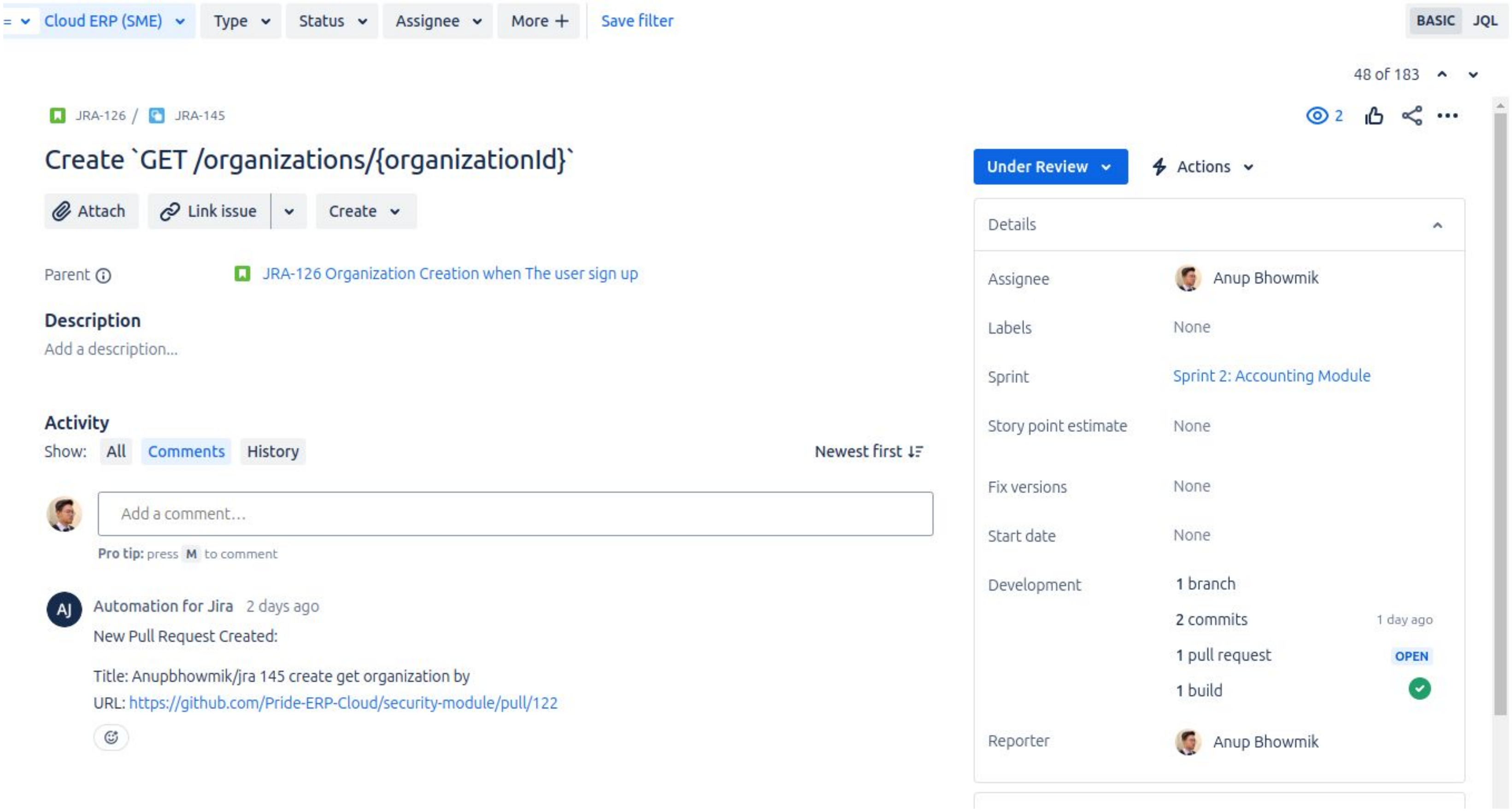
- Sprint Plan
- Github Integration
- Issue Tracking

JIRA



Projects / Cloud ERP (SME)
Timeline
Search timeline
IZ
ME
FA
AR
+4
Status category
Epic
Label
Type
L
AUG
SEP
Sprints
Sprint 2: Accounting...
Releases
JRA-2 User sign-up/sign-in
COMPLETED
JRA-46 Azure Cloud deploym
COMPLETED
JRA-63 Organization Setup
JRA-125 Chart of Accounts and Journal Ei
JRA-150 File Service
JRA-61 subscription
JRA-190 Documentaion for product
JRA-56 MFA
+ Create Epic

JIRA



Cloud ERP (SME)
Type
Status
Assignee
More +
Save filter
BASIC
JQL
48 of 183
JRA-126 / JRA-145
2
Create `GET /organizations/{organizationId}`
Attach
Link issue
Create
Parent
JRA-126 Organization Creation when The user sign up
Description
Add a description...
Activity
Show: All Comments History
Newest first
Add a comment...
Pro tip: press M to comment
AJ
Automation for Jira 2 days ago
New Pull Request Created:
Title: Anupbhowmik/jra 145 create get organization by
URL: https://github.com/Pride-ERP-Cloud/security-module/pull/122
Under Review
Actions
Details
Assignee
Anup Bhowmik
Labels
None
Sprint
Sprint 2: Accounting Module
Story point estimate
None
Fix versions
None
Start date
None
Development
1 branch
2 commits
1 day ago
1 pull request
OPEN
1 build
Reporter
Anup Bhowmik

Design

Data Modeling

API Docs

UI/UX Design

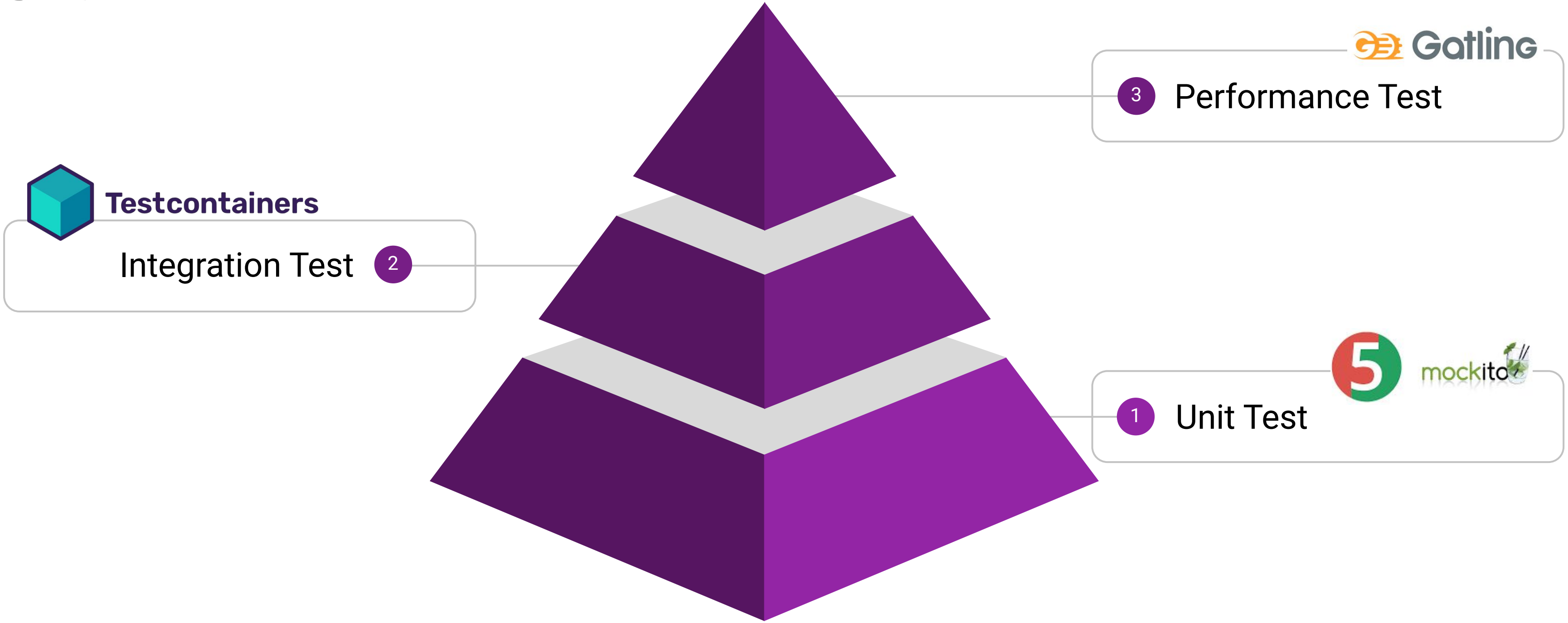
Develop

Backend

Frontend

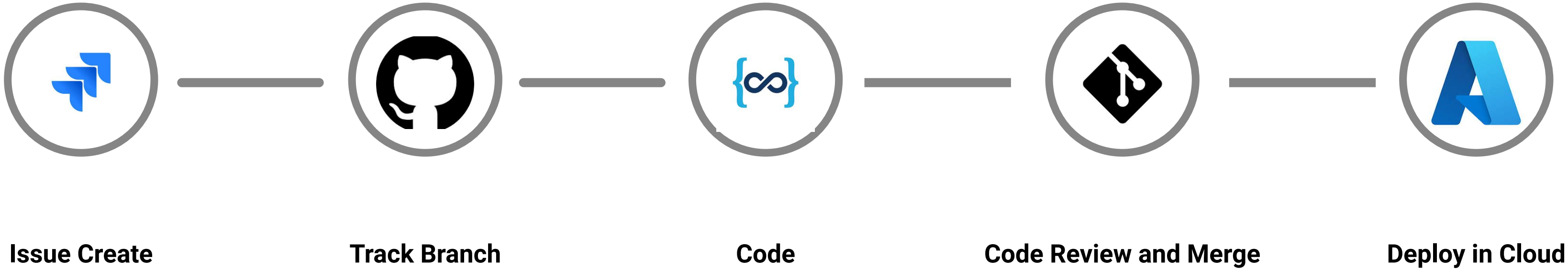
Database

Test

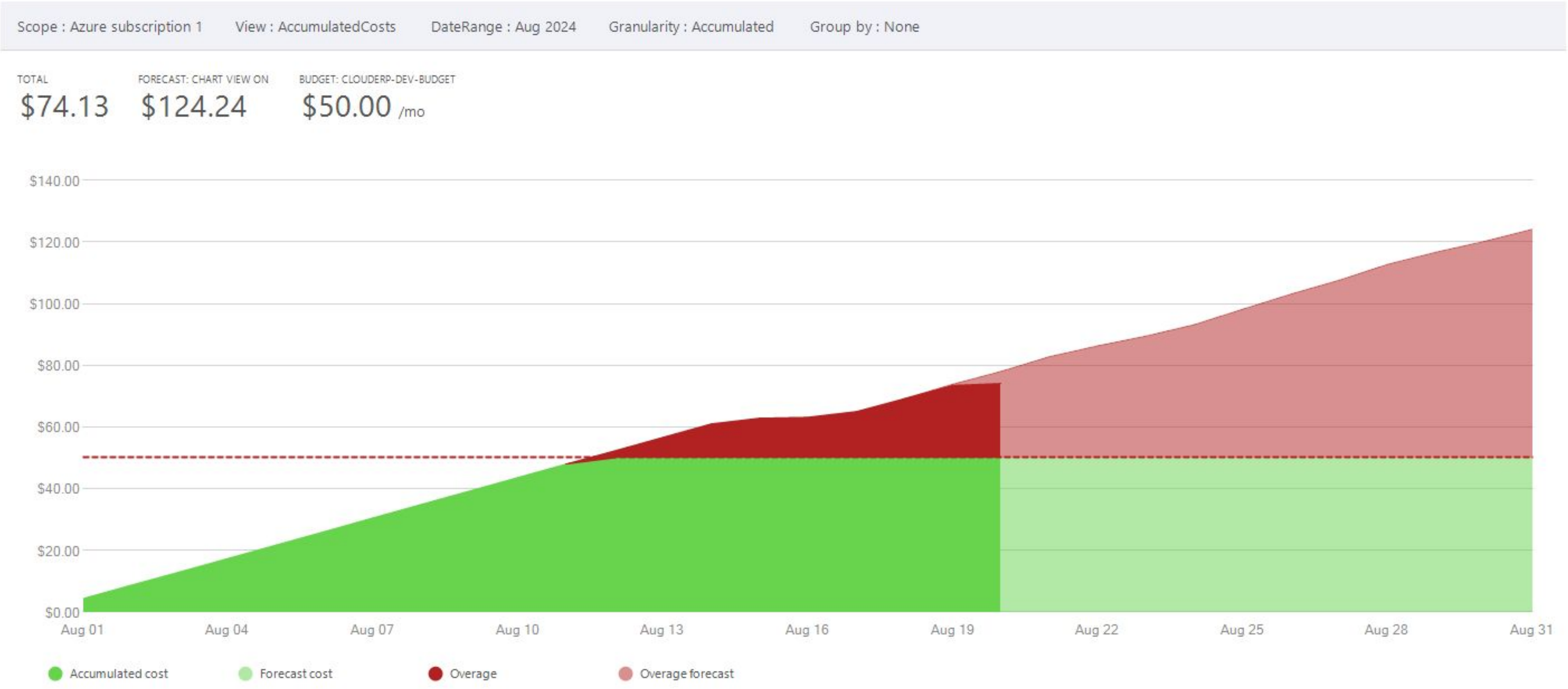


Gatling
3 Performance Test
Testcontainers
Integration Test 2
5
mockito
1 Unit Test

Review & Deploy



Development Costing



Development Costing

Team

Advanced collaboration for individuals and organizations

$ 4 USD per user/month

Continue with Team

Features

- Deployment Branches and secrets
- Branch Protection
- **3,000** CI/CD minutes

Free

Free forever for 10 users

$0

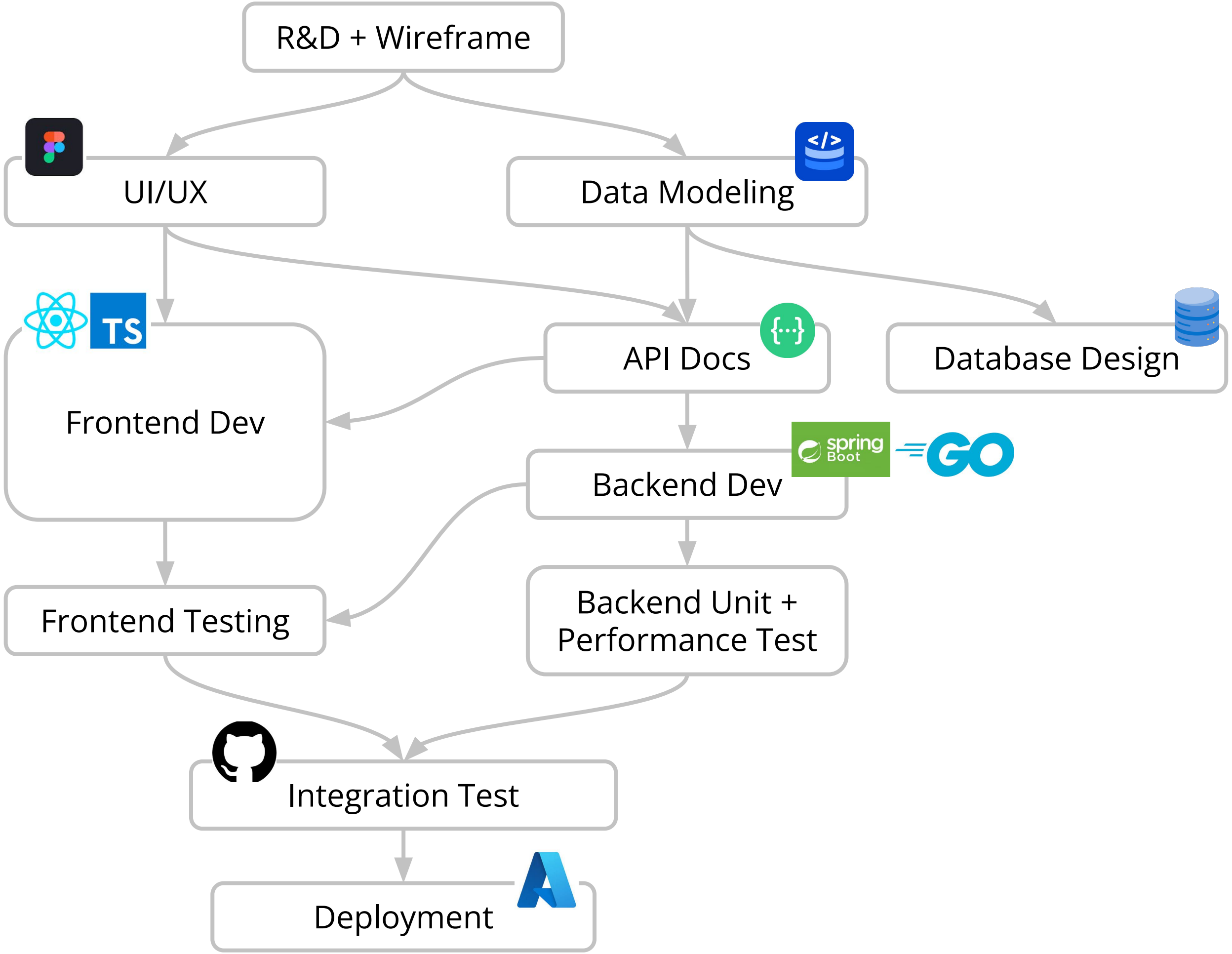
Standard

Everything you need to get started

$7.16

per user / month

Sprint Time Flow



Architecture

- High level System Design
    - Microservice architecture
    - Multi tenant architecture
    - Event driven Architecture
    - Database Perservice
- Interservice communication
    - Async service communication

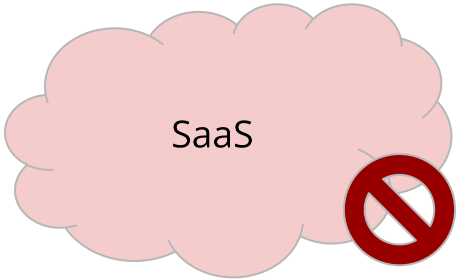
Architecture

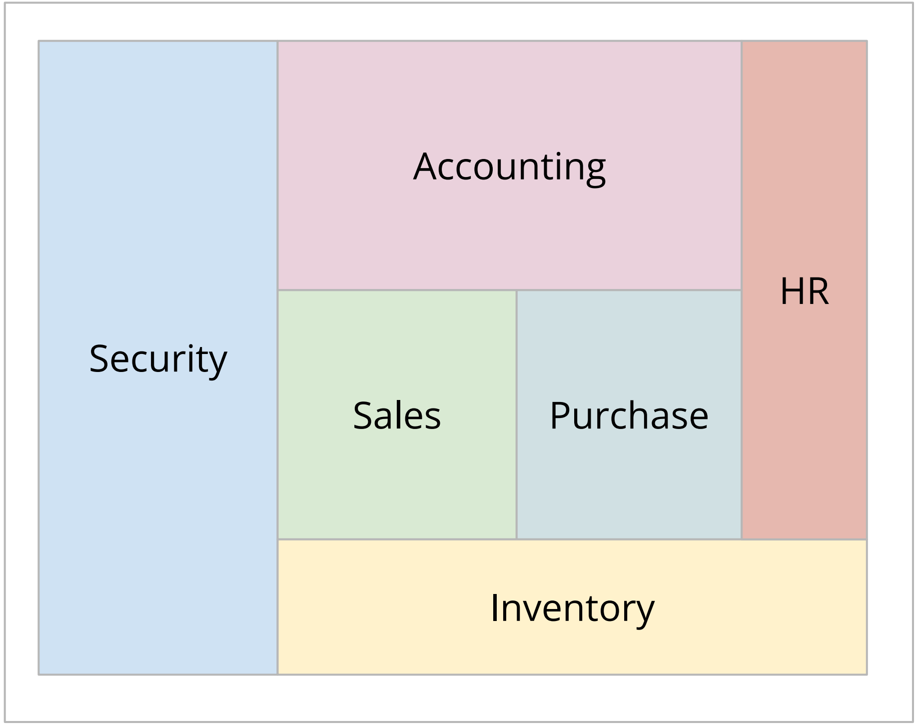
Microservice, Multi-Tenant, SaaS

Monolithic Application

Different Modules -

- Security
- Accounting
- Sales
- Purchase
- HR
- Inventory





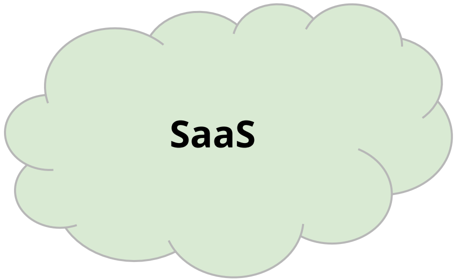
ERP Solution

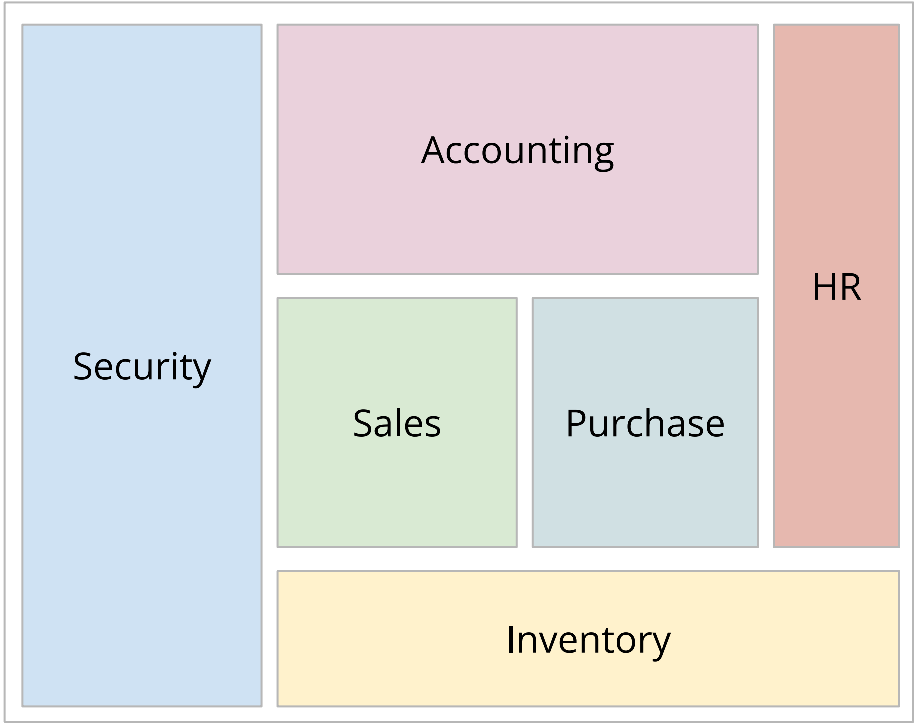
Microservice Application

Some microservices -

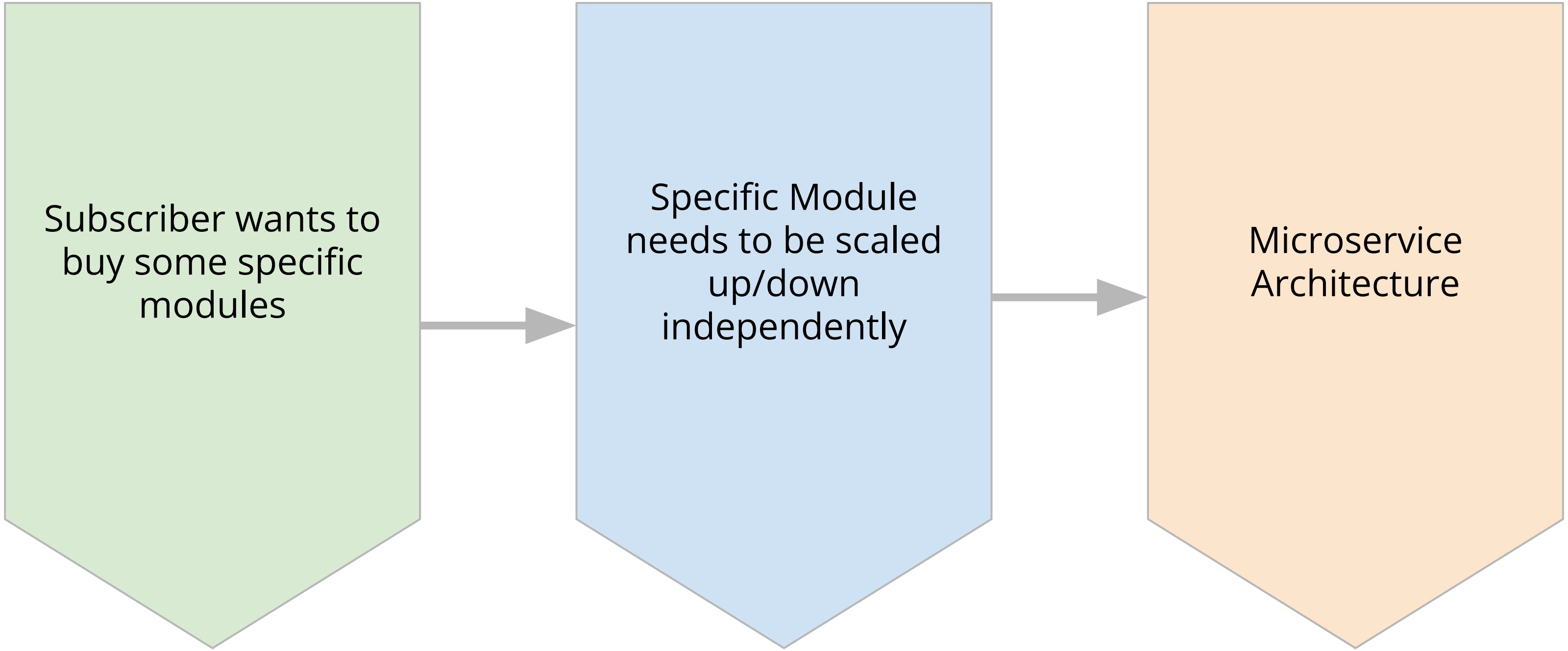
- Security
- Accounting
- Sales
- Purchase
- HR
- Inventory

Control resource usage individually

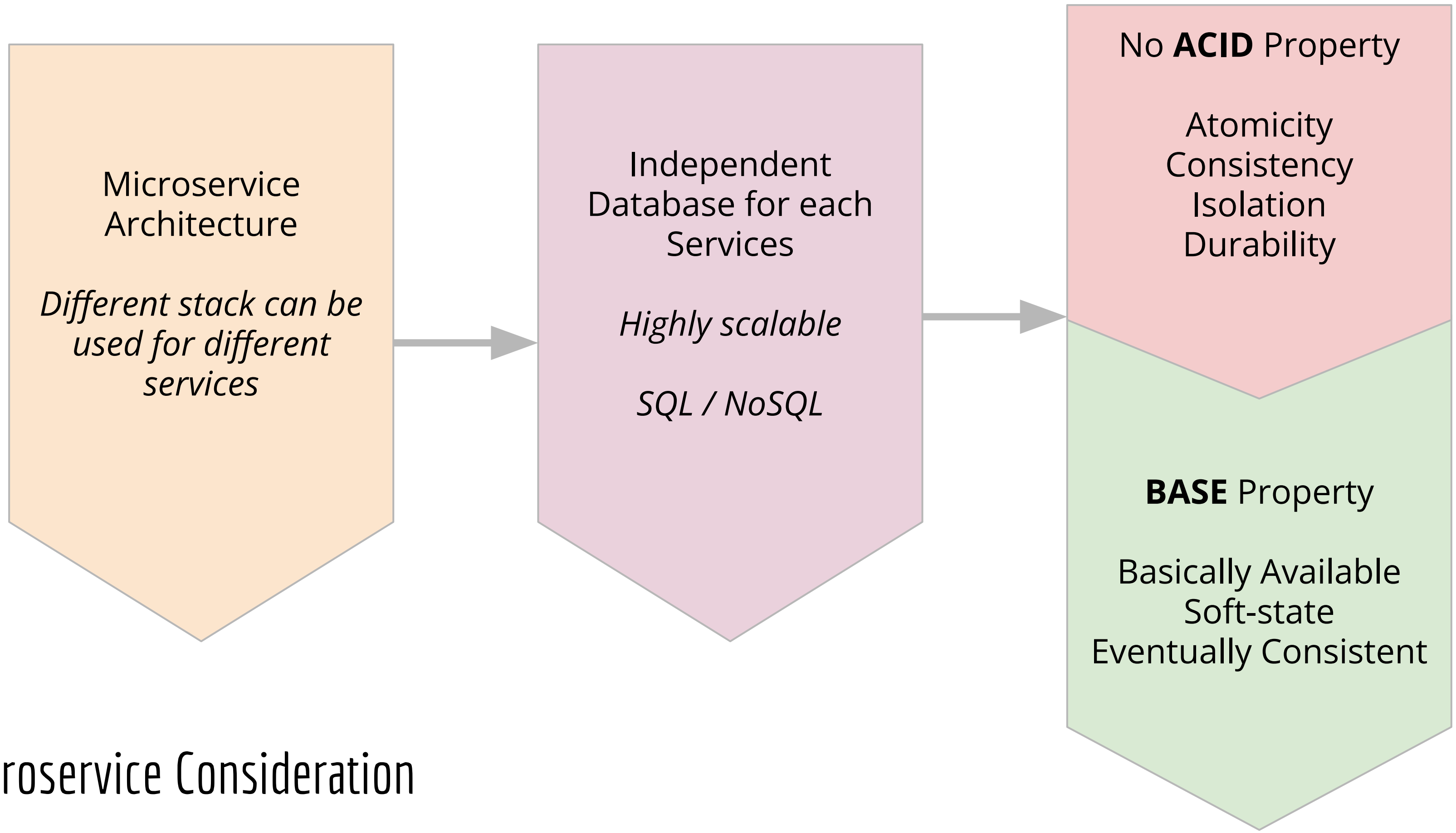




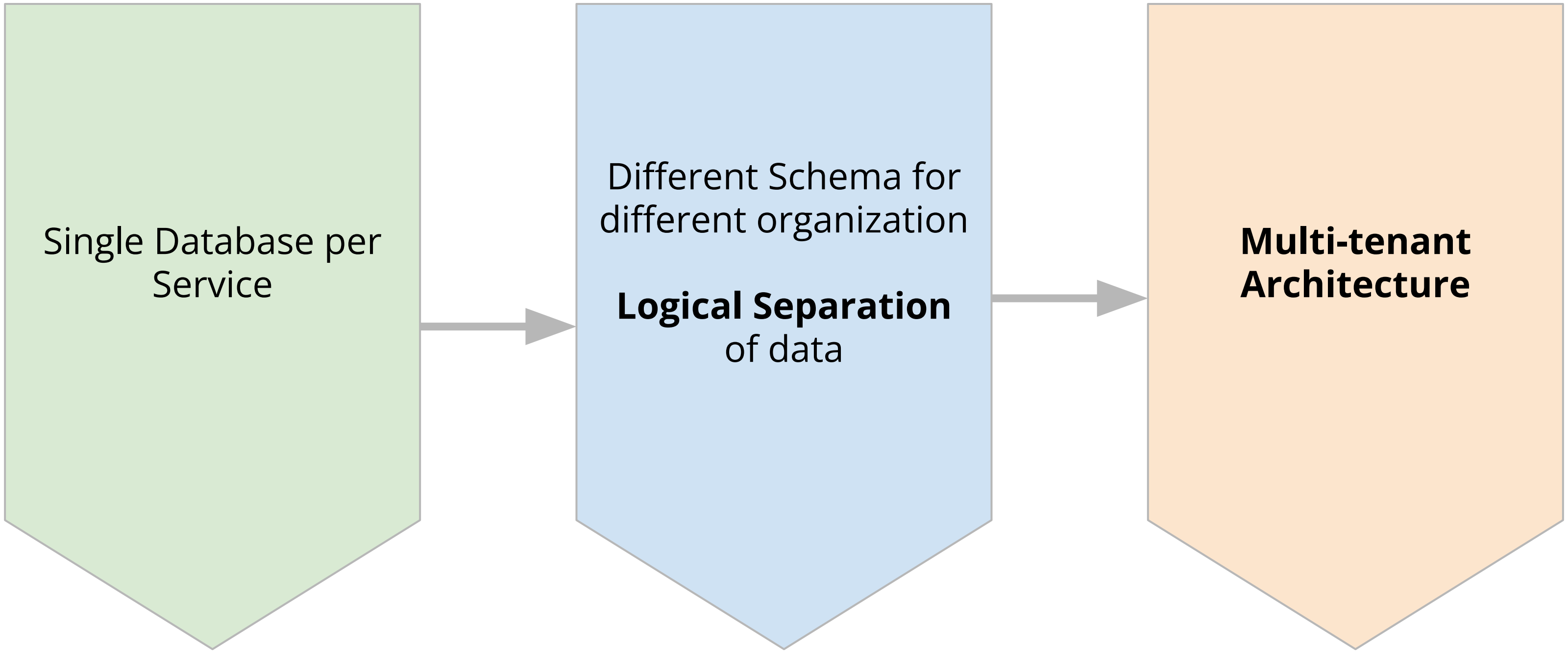
ERP Solution



SaaS Consideration



Microservice Consideration



Different Organization Consideration

SaaS Special Considerations

User can buy any modules independently

- we need to scale each modules independently depending on the subscription

Solution:

- Use of Microservice Architecture
- Each microservice has its own database
- Instead of ***ACID*** (Atomicity, Consistency, Isolation, Durability), uses ***BASE*** (Basically Available, Soft-state, Eventually-consistent)

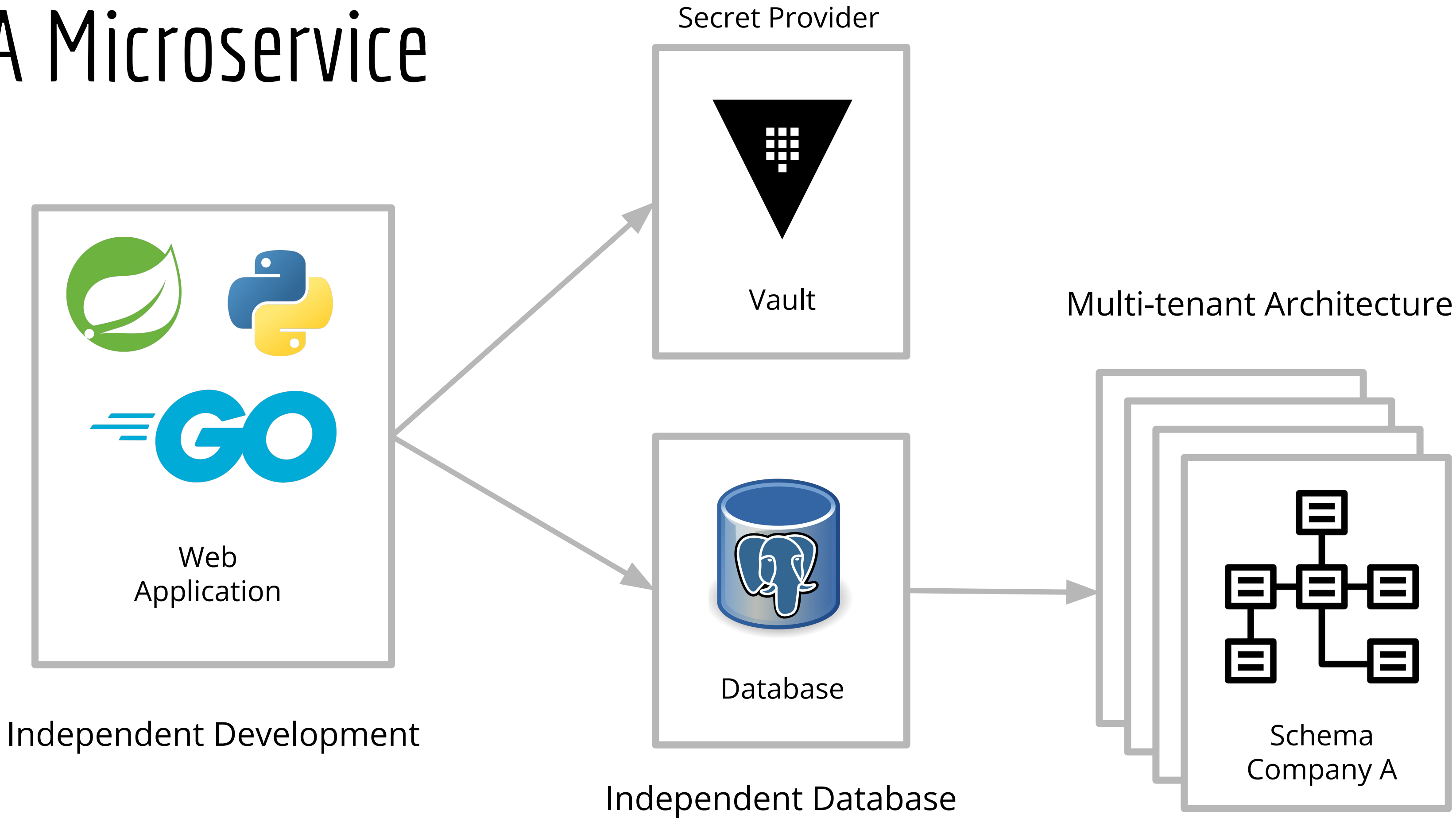
Services are shared by different organizations, What about database?

- Different Schema for different organization in same database
- Logical Separation of data

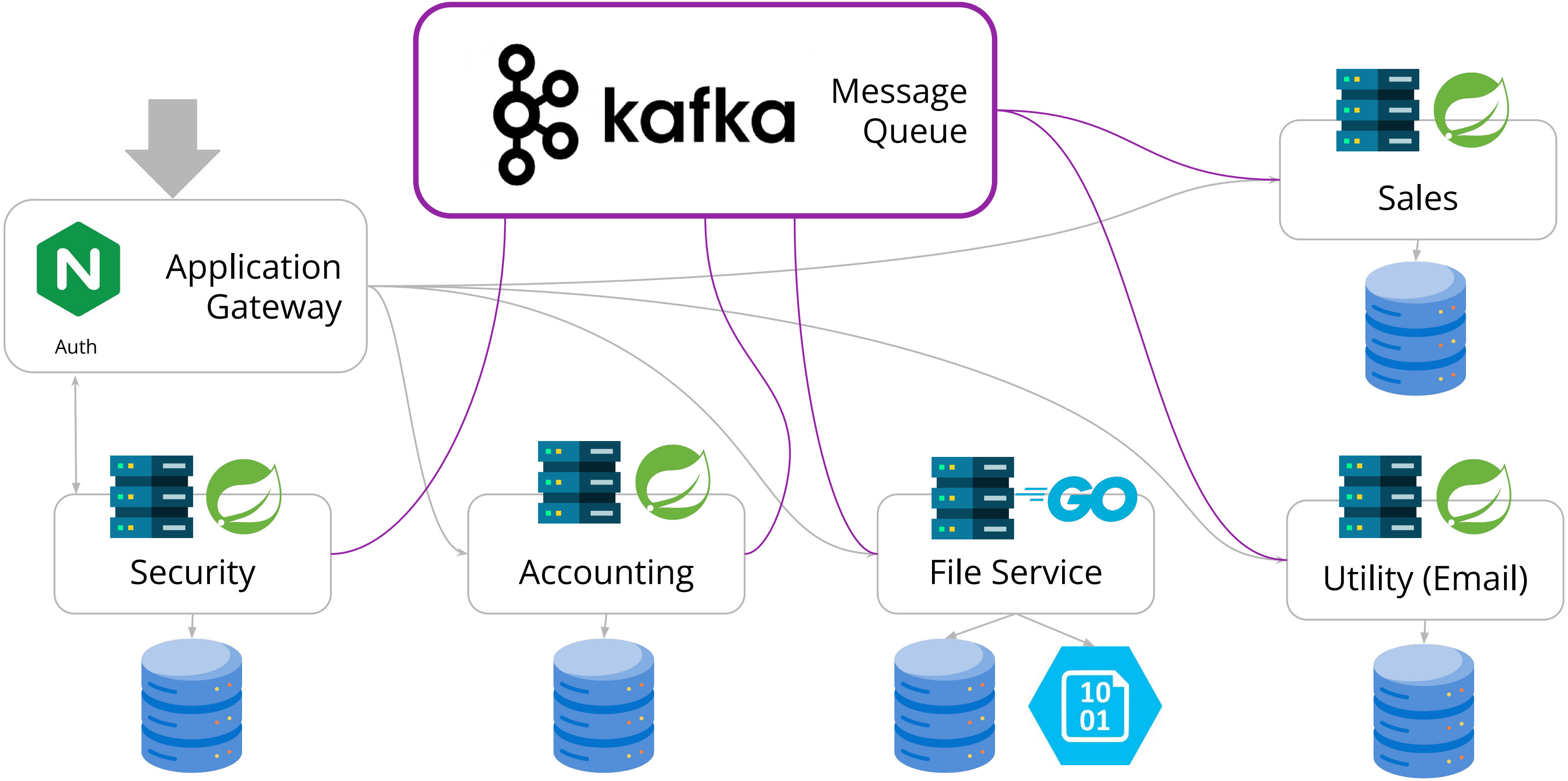
Special Services for common tasks:

- File Service
- Utility Service (Email)

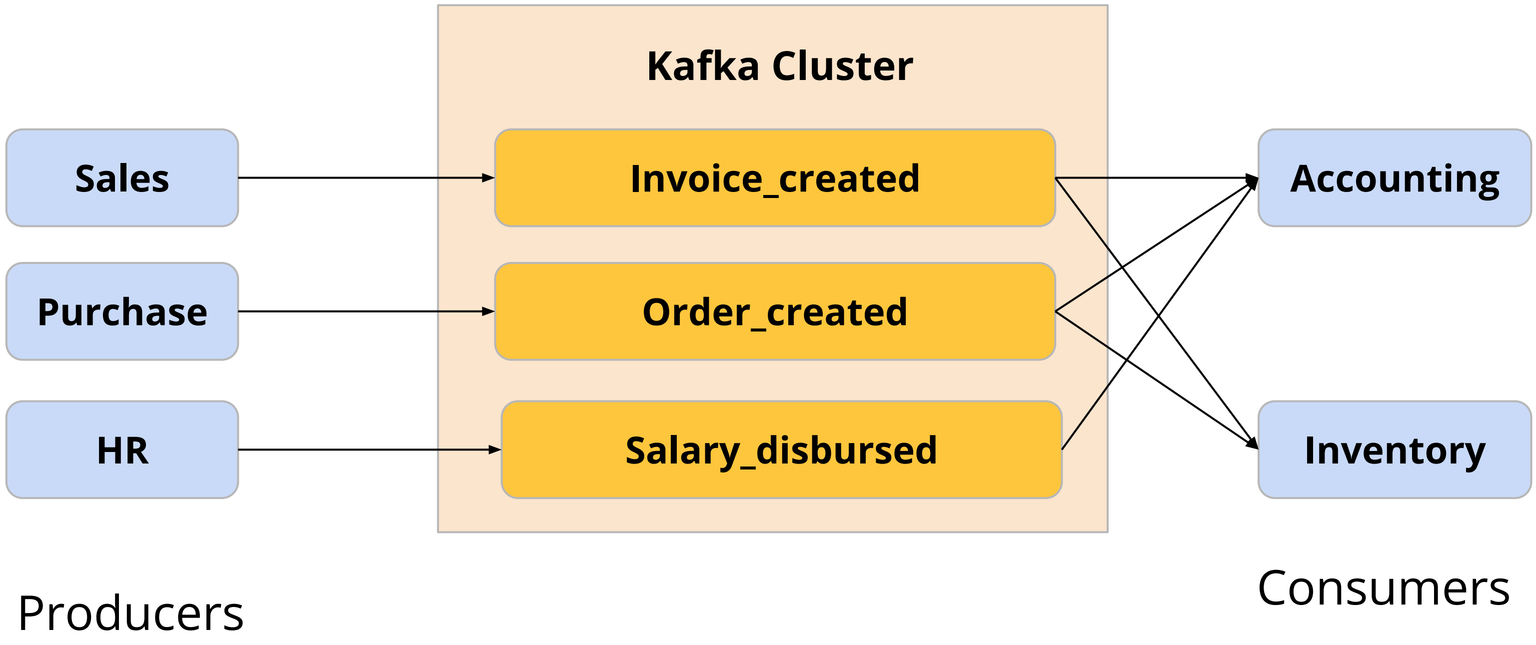
A Microservice



Current Microservices



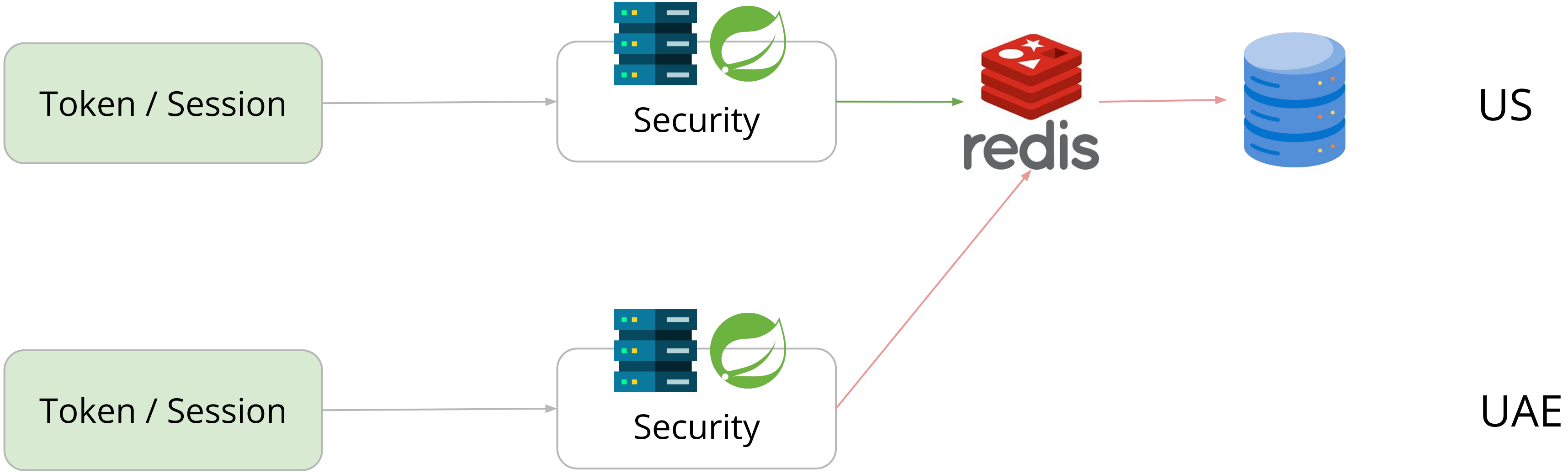
Event Driven Architecture with Kafka



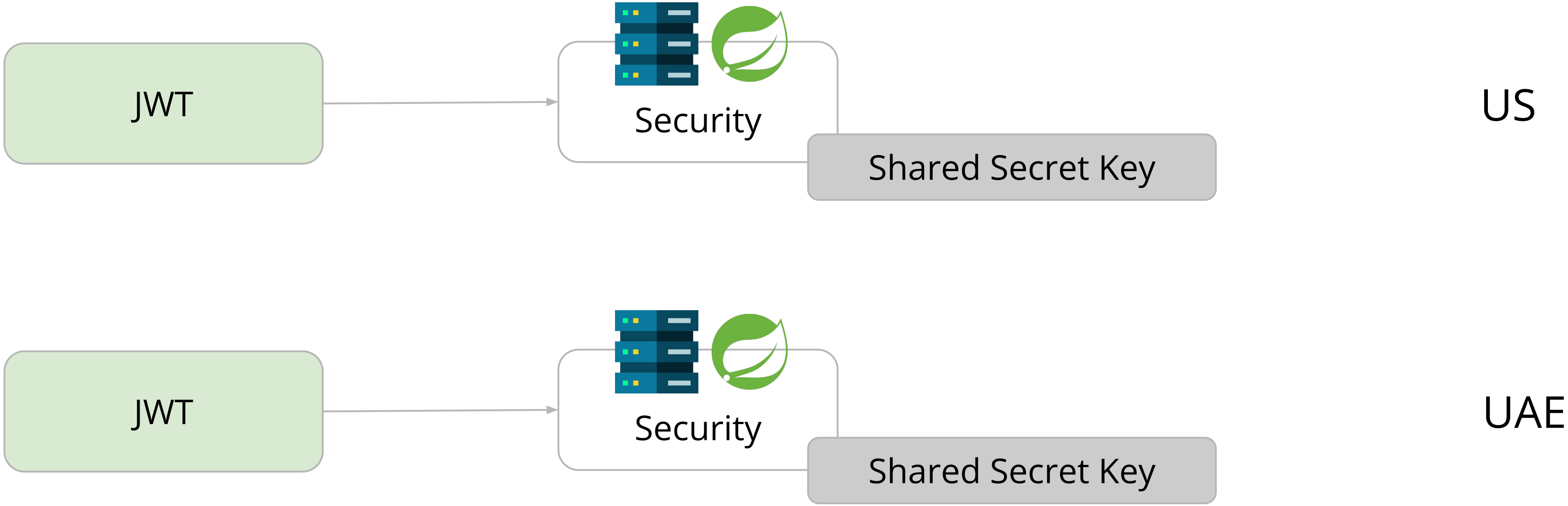
Deep Dive into Authentication #Normal

Deep Dive into Authentication #Low Latency

Deep Dive into Authentication #Distributed



Deep Dive into Authentication #Distributed



Is JWT enough for authentication and authorization?

JWT

Is sent via 'Authentication' Header or as Cookie (which is also a HTTP Header)

Authentication Header Typical Limit is -

8kB

```
{
    "userId": "....",
    "organizationId": "....",
    "role": "member",
    "permissions": ???
}
```

```
Accounting.ManualJournal = ["Create", "Draft", "Read", "Edit", "Approve"]
```

Some Calculation - Permission

There are about **500 permissions** in current PrideERP, considering each permission can have **8 true/false** values, we need -

500 * 8 bits = 500 Bytes binary data

JSON cannot have binary data, we encode the binary data as **base64**

500 Bytes * 133% = 665 Bytes (base64 encoding needs 33% more space)

JWT keeps JSON Payload as base64, so finally space needed in header

665 Bytes * 133% = **885 Bytes** (which can be fitted to 8kB header)

File Service - Golang

File Service uploads file to Azure Blob Storage

Go (Gin) outperforms **2.5x** than Spring Boot due to its better concurrency support.

RPS - Request per Second

[Experiment Link]

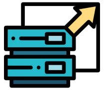
System	RPS
Go (Gin)	3333
Rust (Actix)	2706
Node (Multer)	2353
Node (Koa)	2303
SpringBoot Webflux	2272
Bun (Elysia)	2047
SpringBoot MVC	1306
Python (FastAPI)	769
Deno (Oak)	537

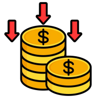
Infrastructure

- Current Hosted solution, why cloud?
- Kubernetes
- Infrastructure description
- Managing infrastructure

Cloud over On Premise


Global Customer Base


Scalability


Cost Efficiency

Kubernetes

Orchestration tool for container

Container → Pod [Instance]

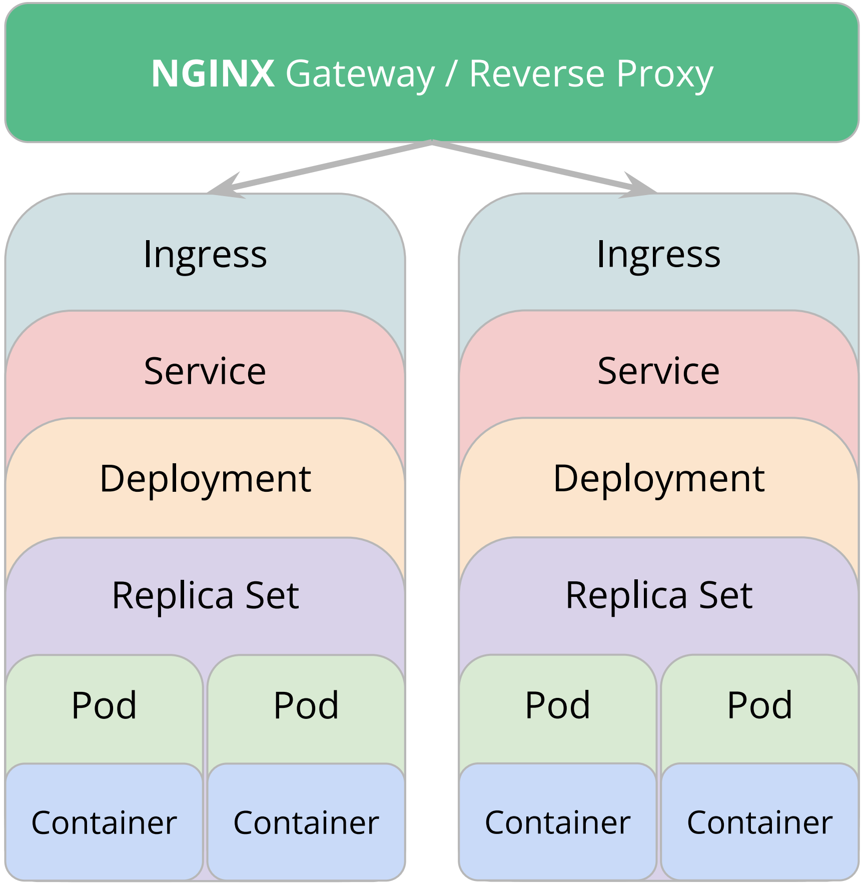
Pod → ReplicaSet [Horizontal Scaling + Fault Tolerant]

ReplicaSet → Deployment [Load Balancer + Vertical Scaling]

Deployment → Service [Service Discovery]

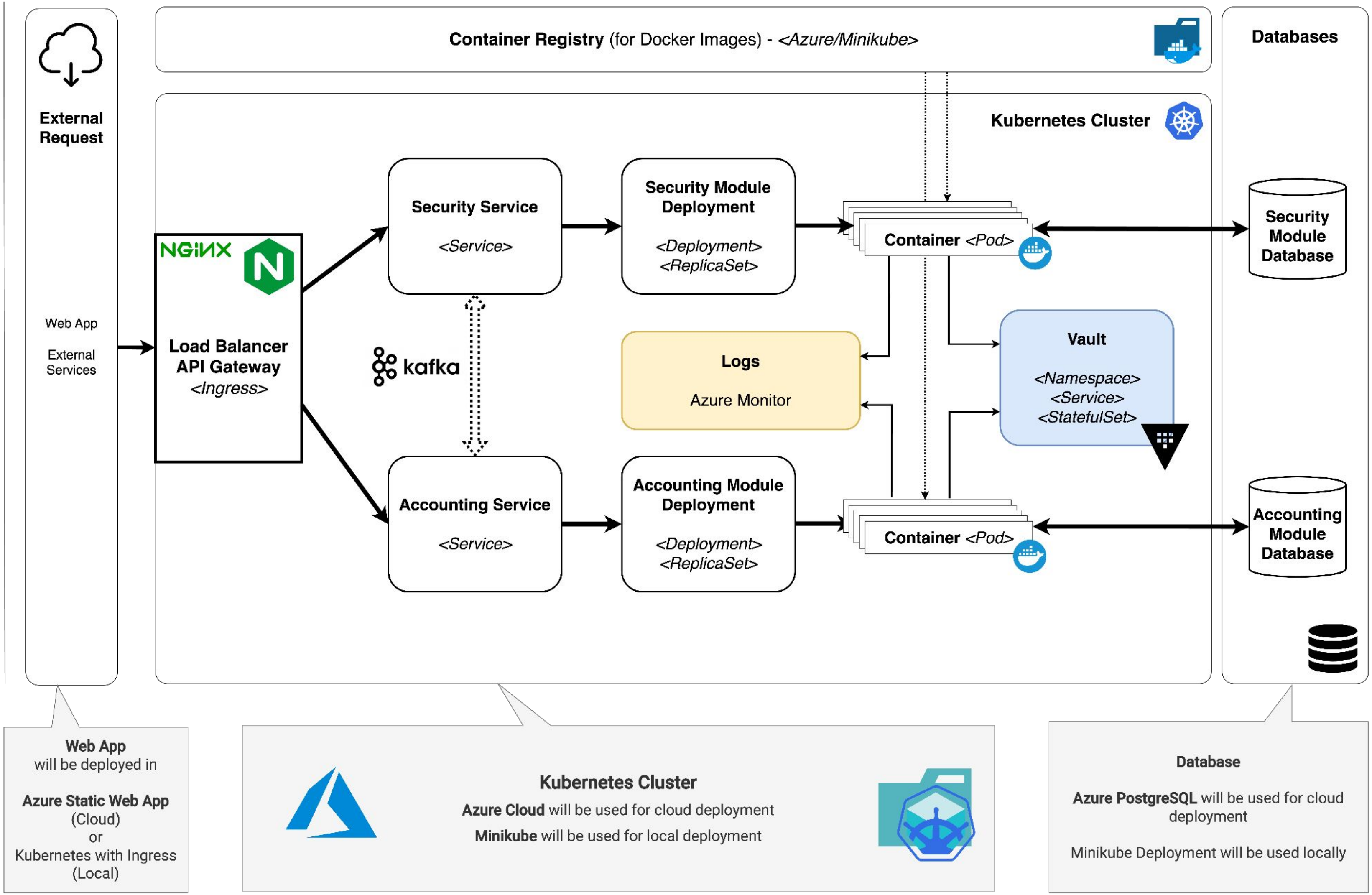
Service → Ingress [Routing + TLS/SSL]

Ingress → Ingress Controller [NGINX Gateway]



Life without Kubernetes

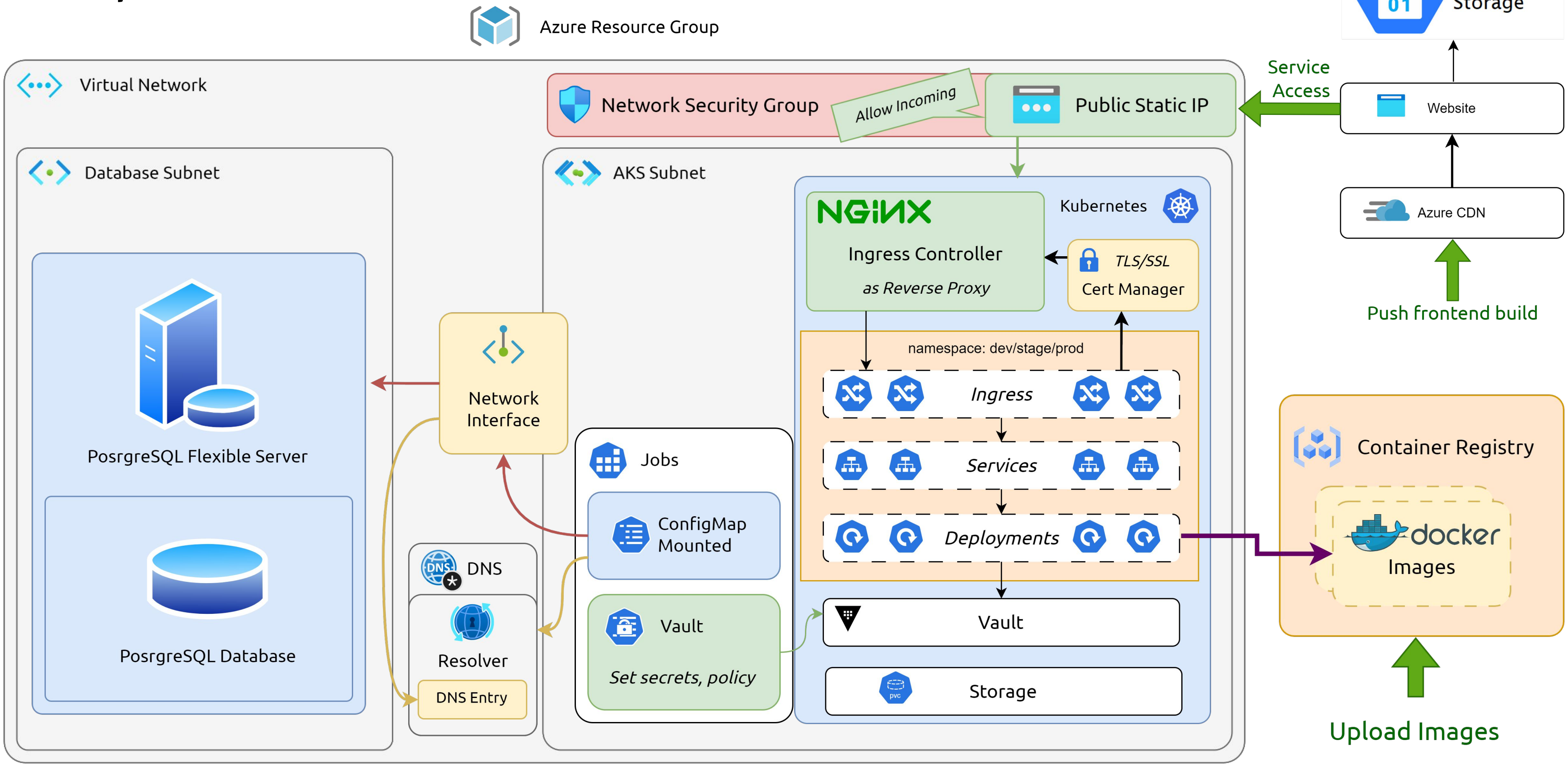
Microservices Concern	Spring Cloud & Netflix OSS	Kubernetes
Configuration Management	Config Server, Consul, Netflix Archaius	Kubernetes ConfigMap & Secrets
Service Discovery	Netflix Eureka, Hashicorp Consul	Kubernetes Service & Ingress Resources
Load Balancing	Netflix Ribbon	Kubernetes Service
API Gateway	Netflix Zuul	Kubernetes Service & Ingress Resources
Service Security	Spring Cloud Security	-
Centralized Logging	ELK Stack (LogStash)	EFK Stack (Fluentd)
Centralized Metrics	Netflix Spectator & Atlas	Heapster, Prometheus, Grafana
Distributed Tracing	Spring Cloud Sleuth, Zipkin	OpenTracing, Zipkin
Resilience & Fault Tolerance	Netflix Hystrix, Turbine & Ribbon	Kubernetes Health Check & resource isolation
Auto Scaling & Self Healing	-	Kubernetes Health Check, Self Healing, Autoscaling
Packaging, Deployment & Scheduling	Spring Boot	Docker/Rkt, Kubernetes Scheduler & Deployment
Job Management	Spring Batch	Kubernetes Jobs & Scheduled Jobs
Singleton Application	Spring Cloud Cluster	Kubernetes Pods


External Request
Web App
External Services
Container Registry (for Docker Images) - <Azure/Minikube>
Kubernetes Cluster
NGINX
Load Balancer API Gateway <Ingress>
Security Service <Service>
Security Module Deployment <Deployment> <ReplicaSet>
Container <Pod>
kafka
Logs
Azure Monitor
Vault <Namespace> <Service> <StatefulSet>
Accounting Service <Service>
Accounting Module Deployment <Deployment> <ReplicaSet>
Container <Pod>
Databases
Security Module Database
Accounting Module Database
Web App will be deployed in Azure Static Web App (Cloud) or Kubernetes with Ingress (Local)
Kubernetes Cluster
Azure Cloud will be used for cloud deployment
Minikube will be used for local deployment
Database
Azure PostgreSQL will be used for cloud deployment
Minikube Deployment will be used locally

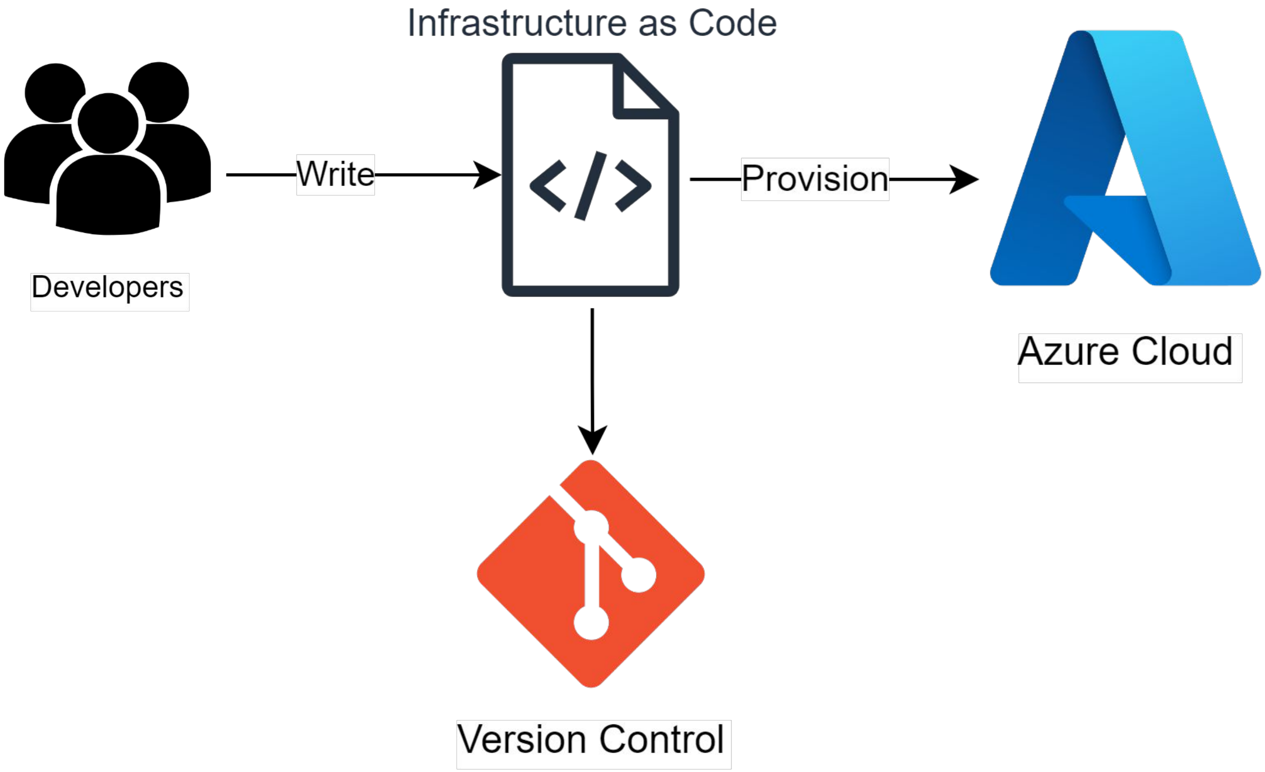
Infrastructure

Resource Provision, State management, Version controlling

Infrastructure



Infrastructure as Code (IaC)



IaC Tool

Terraform → OpenTofu

THE LINUX FOUNDATION PROJECTS

The open source infrastructure as code tool.

Previously named OpenTF, OpenTofu is a fork of Terraform that is open-source, community-driven, and managed by the Linux Foundation.

Infrastructure Provisioning

🛠️ Initialize OpenTofu

Run the following command to initialize OpenTofu, which will download the necessary providers and modules:

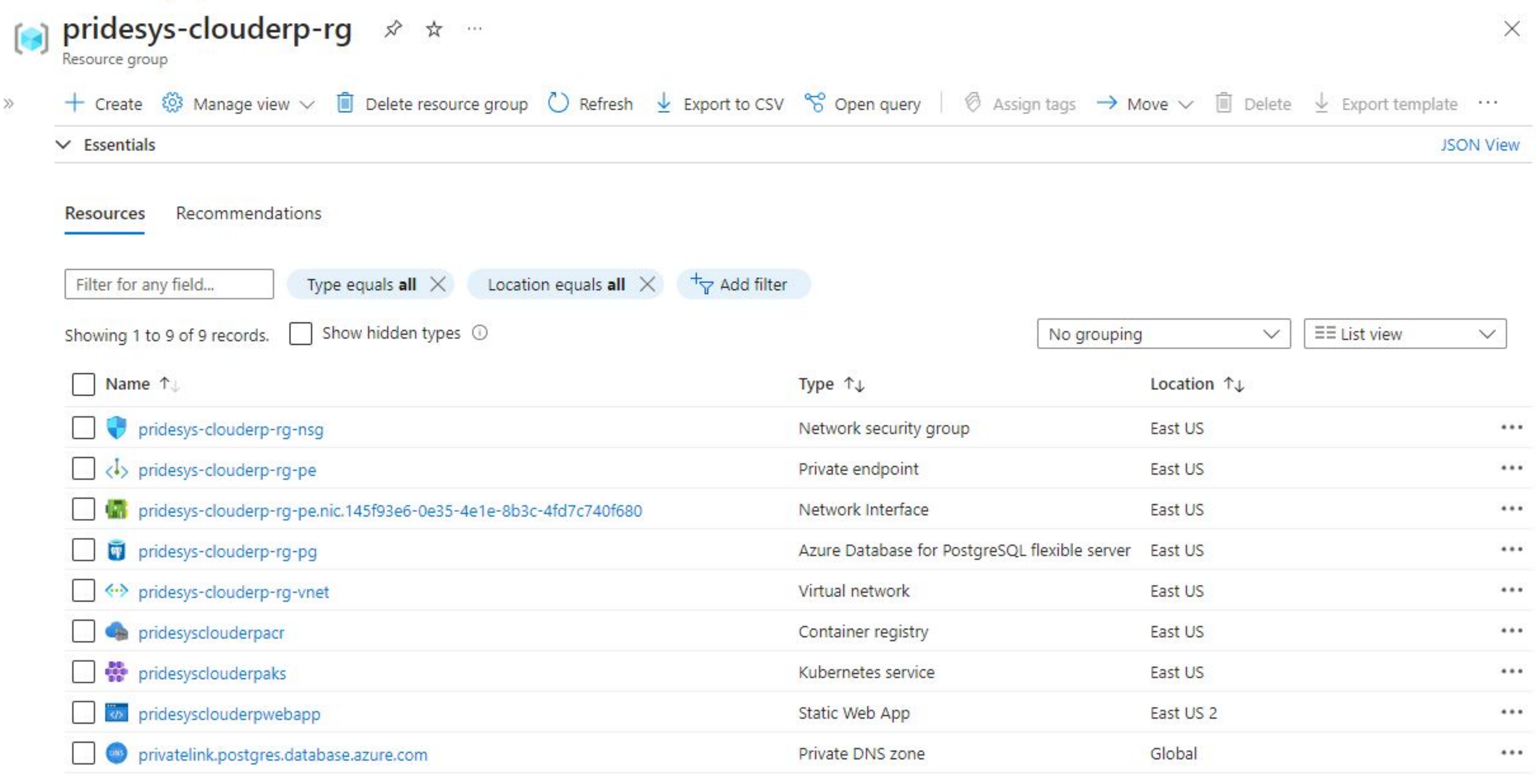
```
tofu init --backend-config=backend.conf
```

🚀 Provision Infrastructure

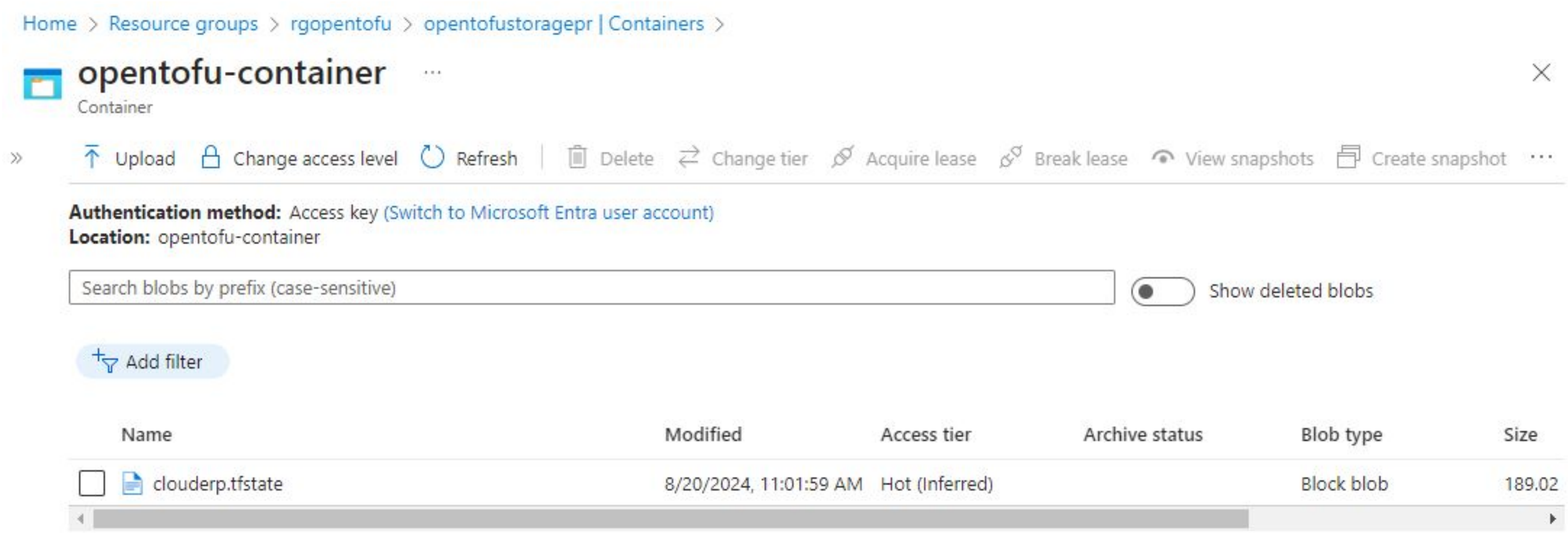
To see the changes that OpenTofu will perform, run:

```
tofu apply -auto-approve -target="module.infra" -target="module.config" && tofu apply -auto-approve
```

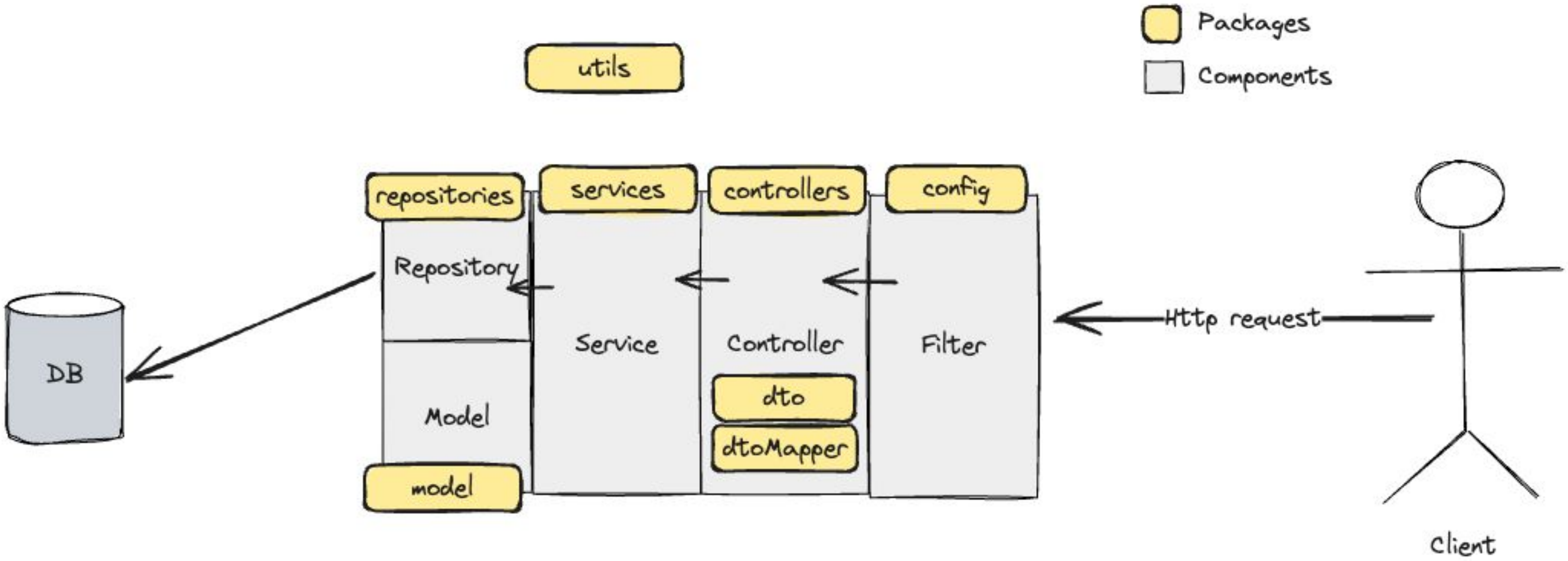
Infrastructure in Azure



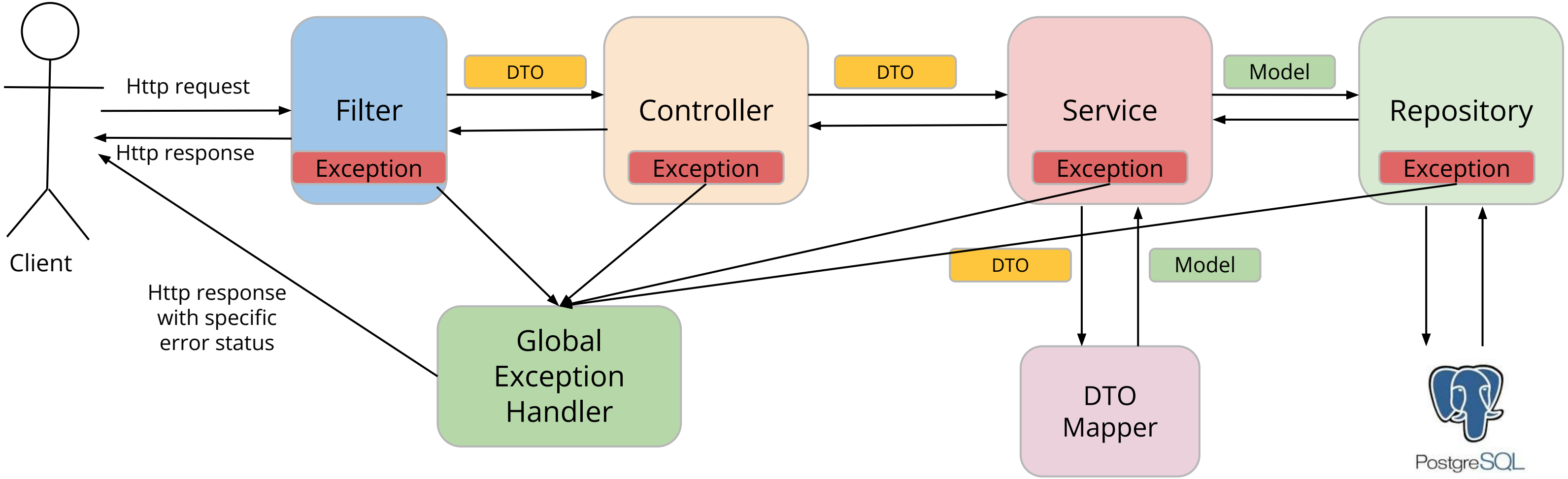
Infrastructure State Management



Service Development (Spring Boot)



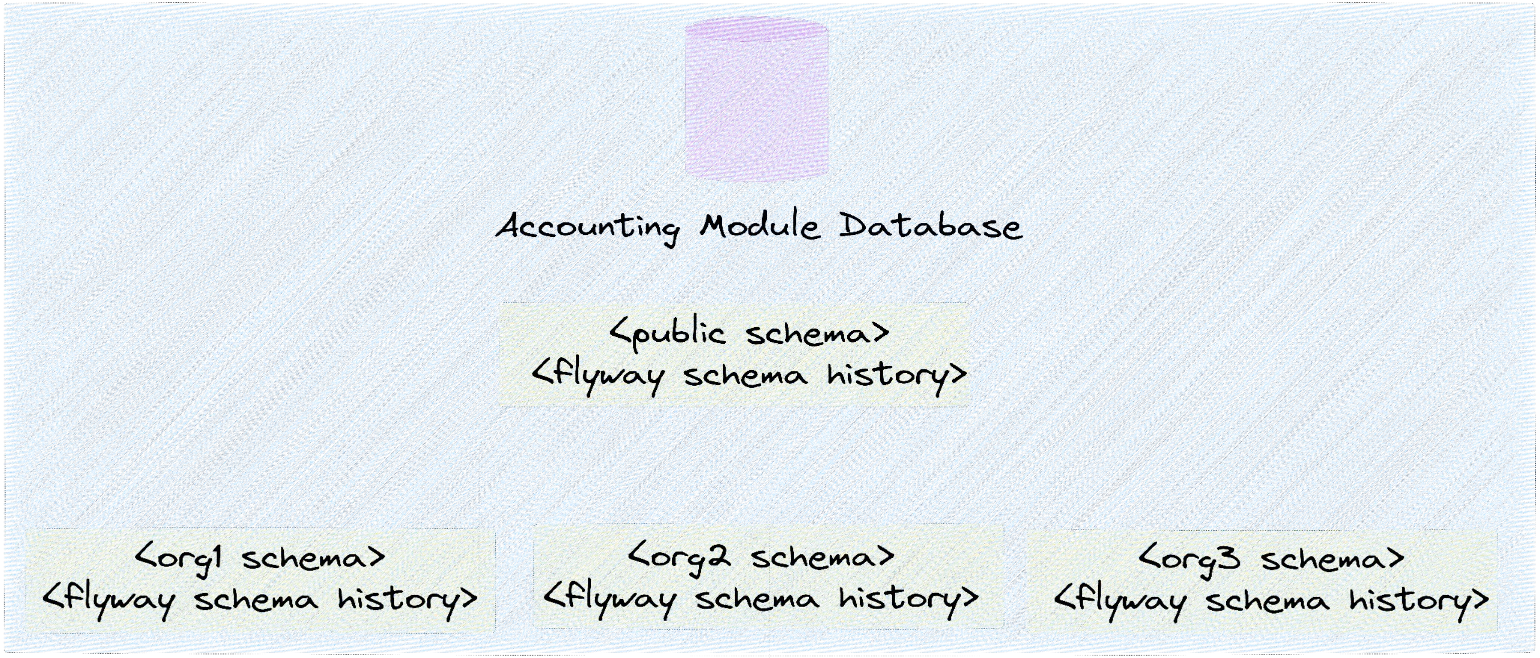
Service Development (Spring Boot)



Project Structure

```
src
├── main
│   ├── java
│   │   └── com.pridesys.clouderp.security
│   │       ├── config
│   │       ├── controller
│   │       ├── dto
│   │       ├── dtoMapper
│   │       ├── exception
│   │       ├── model
│   │       ├── repositories
│   │       ├── service
│   │       ├── types
│   │       └── utils
│   └── resources
│       ├── db
│       └── templates
└── test
    ├── java
    │   └── com.pridesys.clouderp.security
    │       ├── controller
    │       ├── repository
    │       └── service
    └── resources
        └── templates
```

Multi-tenant Database



Flyway Schema History


```
[
    {
        "installed_rank":0,
        "version":null,
        "description":"<< Flyway Schema Creation >>",
        "type":"SCHEMA",
        "script":"\"thisisa234542vdef\"",
        "checksum":null,
        "installed_by":"pridesys",
        "installed_on":"2024-08-19 17:41:16.016356",
        "execution_time":0,
        "success":true
    },
    {
        "installed_rank":1,
        "version":"1",
        "description":"init",
        "type":"SQL",
        "script":"V1__init.sql",
        "checksum":-2040262741,
        "installed_by":"pridesys",
        "installed_on":"2024-08-19 17:41:16.020123",
        "execution_time":27,
        "success":true
    }
]
```

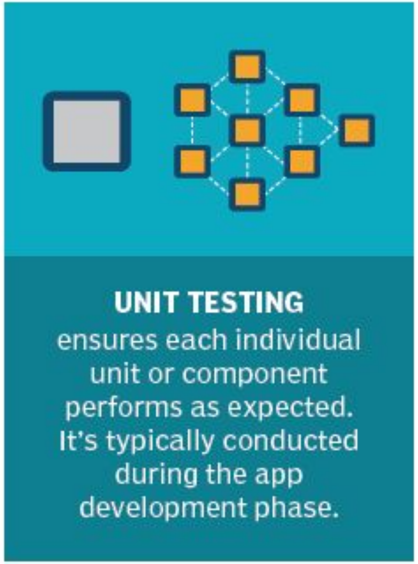
DB Migration Structure

```
src
├── main
│   ├── resources
│   │   └── db
│   │       ├── migration
│   │       │     └── V1__init.sql
│   │       │     ├── V2__add_updatedat_col.sql
│   │       ├── tenants
│   │             └── V1__init.sql
│   │             ├── V2__add_journal_table.sql
│   └── resources
│       └── templates
└── test
```

Tests

- Unit
- Integration
- Performance
- Code coverage

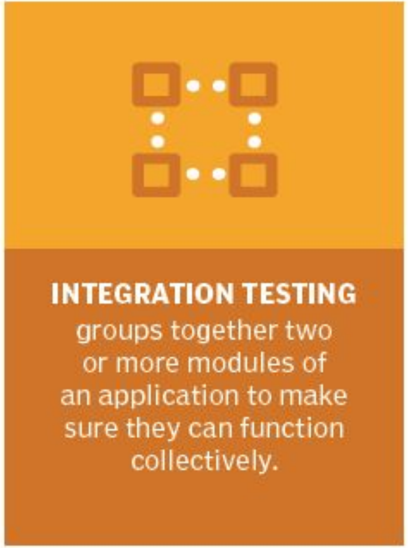
Unit Testing



mockito

JUnit 5

Integration Tests



Testcontainers

Performance Testing

Gatling

Code Coverage

JACOCO
Java Code Coverage

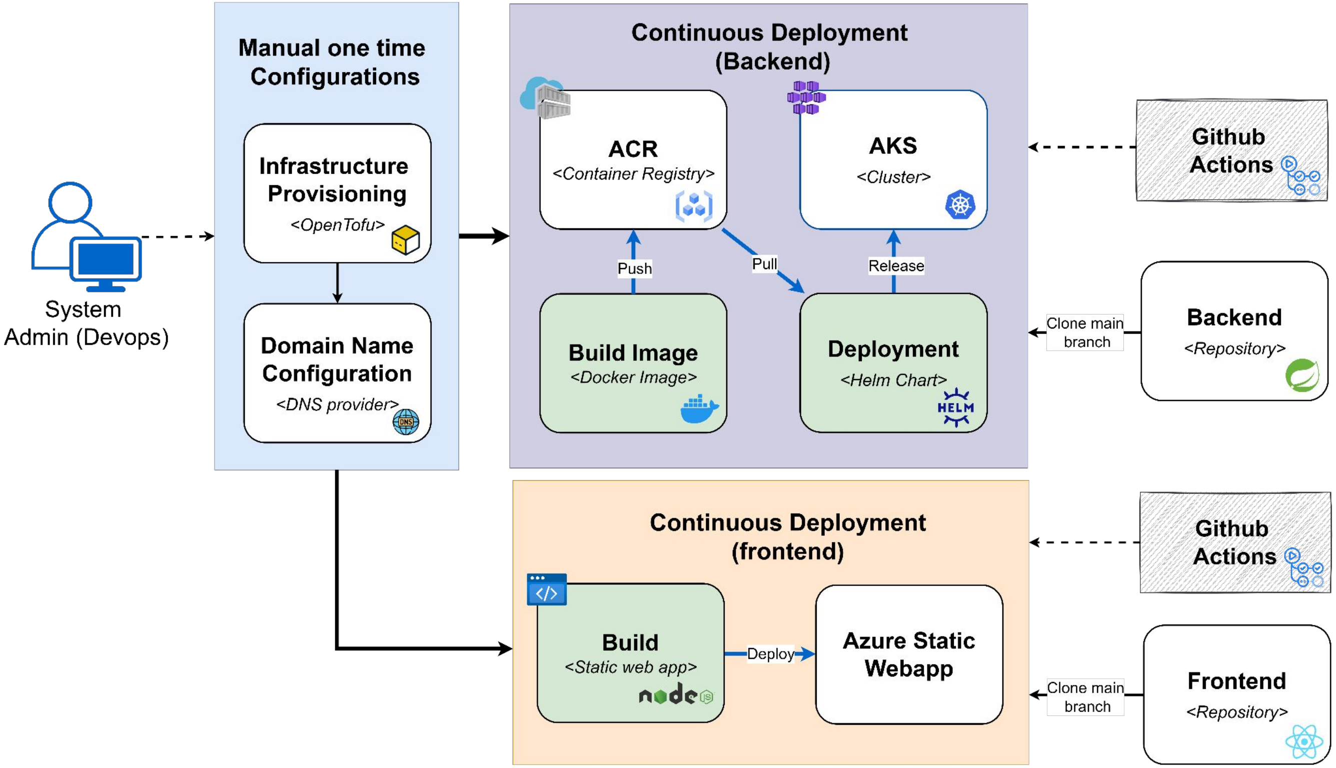
Devops

CI/CD

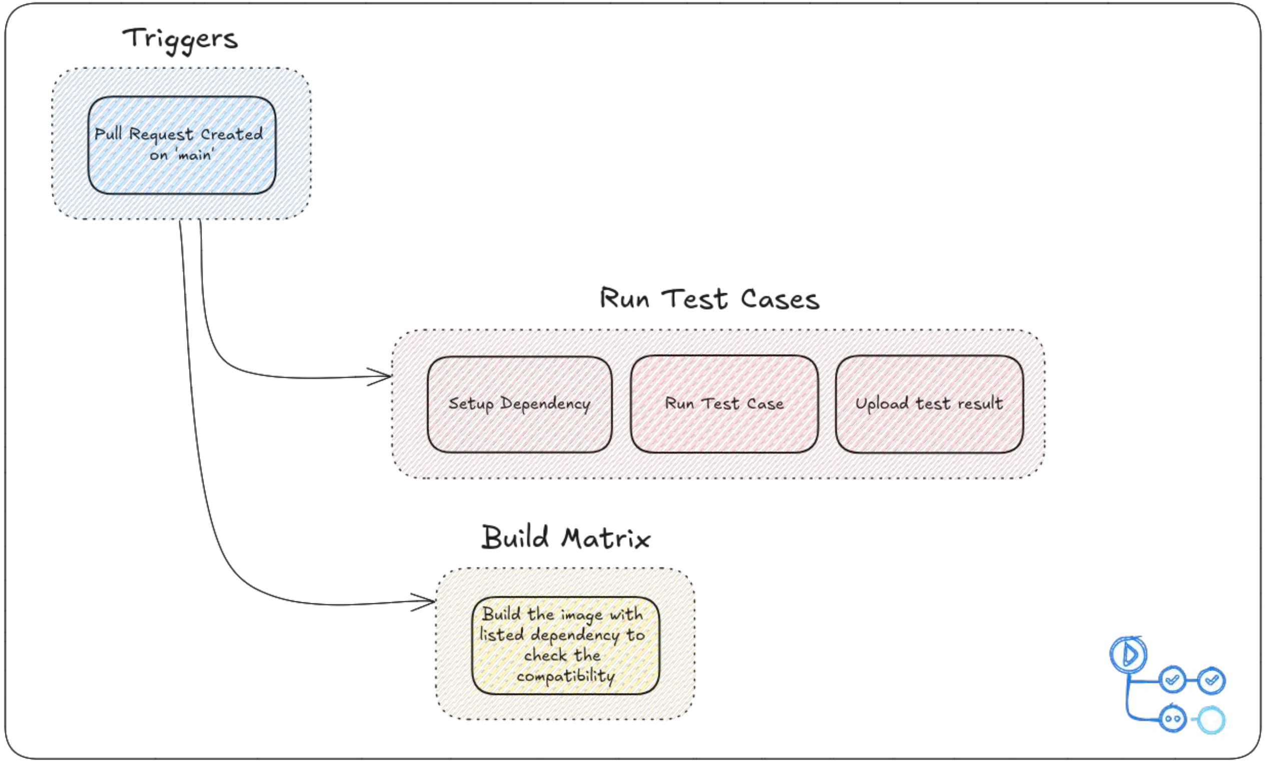
Devops

- Continuous Integration
- Continuous Deployment
- Automation with external software
  - Automated update to JIRA
  - Notification to Chat solution

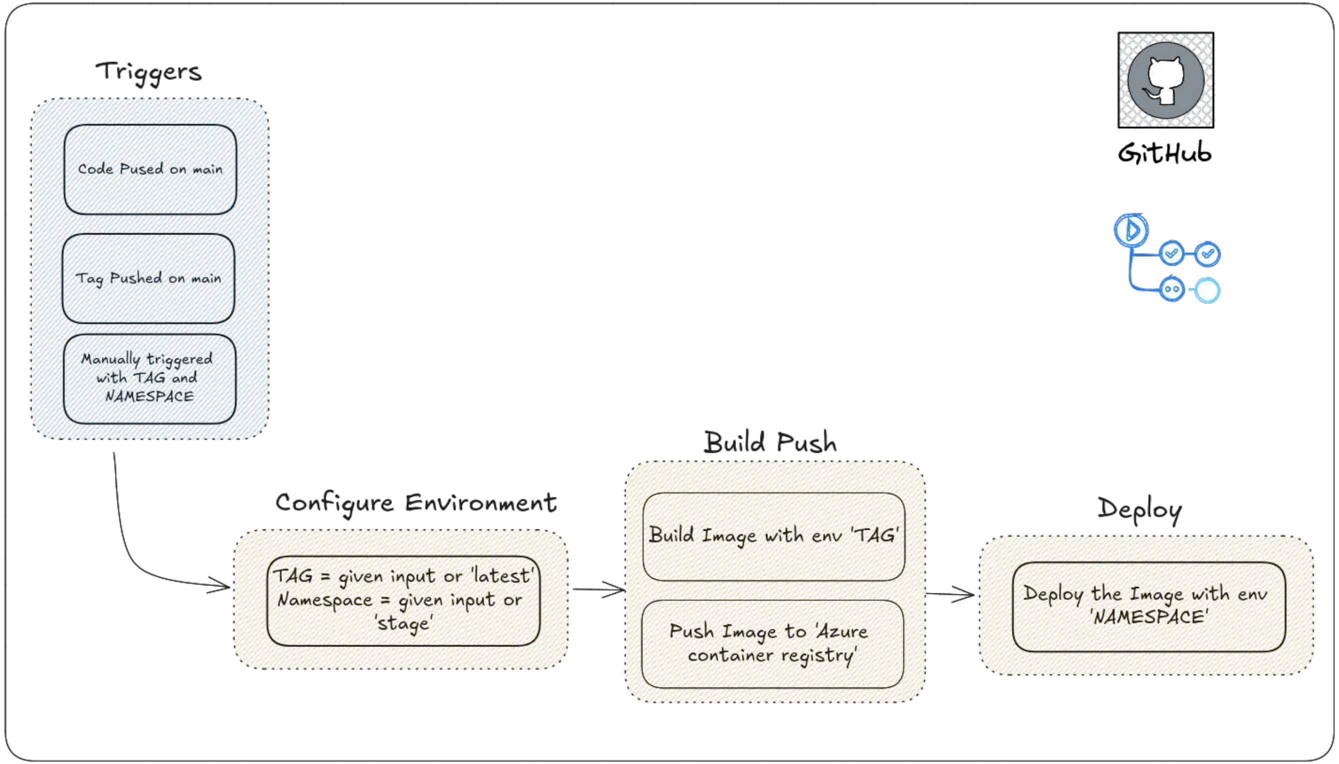
DevOps Lifecycle



CI Workflow


Triggers
Pull Request Created on 'main'
Run Test Cases
Setup Dependency
Run Test Case
Upload test result
Build Matrix
Build the image with listed dependency to check the compatibility

CD workflow


GitHub
Triggers
Code Pused on main
Tag Pushed on main
Manually triggered with TAG and NAMESPACE
Configure Environment
TAG = given input or 'latest'
Namespace = given input or 'stage'
Build Push
Build Image with env 'TAG'
Push Image to 'Azure container registry'
Deploy
Deploy the Image with env 'NAMESPACE'

Automation in JIRA

- When a branch is created □ Move issue to ‘IN PROGRESS’
- When all child issue are completed □ Close the parent issue
- When pull request is merged □ Move the issue to ‘DONE’
- When commit is created □ Move issue to ‘IN PROGRESS’
- When pull request is created □ Move issue to ‘UNDER REVIEW’
- When sub task is created □ Automatically assign subtask from parent issues’ assignee

Notification to Chat solution

- Whenever any sort of automation is triggered Notify about the automation update in one channel.

Current Usage of automation

- In JIRA:
  - Today current usage: 73
  - Limit in every month: 100
- In GITHUB:
  - Last month’s usage: around 1100 minutes.
  - Limit in every month: 3000

Documentation

- Let’s see all our documents: Documentation
- Let’s see our Accounting Module API Documentation: API Doc

Any Questions?